Sessió 9

Desenvolupament Web Full Stack (React, Node, Mongo)
Curs 2025-2026

Pràctica: E-commerce. Sessió 9

1. Dates....3
2. Objectius....3
3. Feina prèvia abans de la sessió....3
Avantatges de l'estratègia avançada d'autenticació amb JWT (Access + Refresh + Rotació)....3
1. Seguretat millorada....3
2. Sessions sense estat (stateless)....3
3. Rotació segura del refresh token....4
4. Protecció davant d'atacs de reutilització (replay attacks)....4
5. Control i revocació....4
6. Millor experiència d'usuari....4
4. Feina durant la sessió i abans de la següent sessió....5
4.1 Autenticaciò amb JWT + Token Refresh y Rotació Segura....5
Objectius:....5
- Implementar login amb generació de access token i refresh token....5
- Mantenir la sessió activa amb rotació de refresh tokens....5
- Protegir rutes amb middleware que validi els tokens....5
- Millorar la seguretat per a evitar atacs de reutilització (token theft)....5
Com implementar un sistema de login amb tokens JWT, mantenir la sessió activa amb rotació segura i protegir les rutes....5
1. Generar un access token i un refresh token en iniciar sessió....5
2. Mantenir la sessió activa amb rotació segura de tokens de refresc....5
3. Protegir les rutes amb middleware que valida els tokens....6
4. Millorar la seguretat contra atacs de reutilització (robo de tokens)....6
4.3 Implementar....7
4.4 Crear l'esquema....7
4.5 Controlador....7
4.6 Service....7
S'encarrega d'interactuar amb la base de dades, xifrar contrasenyes, generar tokens, etc....7
4.7 Routes "routes/usuariRoutes.js"....8
4.8 Executar consultes Postman....8
4.9 Comprovar MongoDB....8
Referències....9
Apèndixs....9
A. Comandes útils de Node/Nodemon....9
B. Fitxer .gitignore recomanat....9
C. Validacions habituals en Mongoose....9
D. Errors comuns i depuració....10
Lliurable de la Sessió 8....10

1. Dates

- 09/03 - 18/03

2. Objectius

Pes	Preferent?	Funcionalitat
0,3	✓	Sistema d'autenticació complet amb JWT
0,3	✓	Implementar registre i login d'usuaris
0,2	✓	Implementar refresh token i logout
0,2	✓	Implementar control d'accés amb RBAC

3. Feina prèvia abans de la sessió

- Revisar conceptes bàsics d'autenticació i autorització.
- Entendre què és JWT (JSON Web Token).
- Revisar el model User creat a la sessió anterior.
- Llegir breument què és RBAC (Role Based Access Control).

4. Feina durant la sessió i abans de la següent sessió

4.1 Instal·lació de dependències

Instal·lar les llibreries necessàries per a l'autenticació:

```
npm install jsonwebtoken bcryptjs
```

Aquestes llibreries serviran per:

- **bcryptjs → xifrar contrasenyes**
- **jsonwebtoken → generar tokens JWT**

4.2 Registre d'usuaris

Crear controlador `authController.js`.

Endpoint:

POST /api/auth/register

Flux:

1. Rebre dades de l'usuari.
2. Verificar que l'email no existeix.
3. Hashejar la contrasenya.
4. Guardar usuari a MongoDB.

4.3 Login amb JWT

Endpoint:

POST /api/auth/login

Flux:

1. Buscar usuari per email.
2. Comparar contrasenya amb bcrypt.
3. Generar **access token**.
4. Generar **refresh token**.

4.4 Refresh token

Endpoint:

POST /api/auth/refresh

Permet generar un nou access token sense tornar a fer login.

Flux:

1. L'usuari envía refresh token.
2. Es vàlida.
3. Es genera un nou access token.

4.5 Logout

Endpoint:

POST /api/auth/logout

Opcions possibles:

- eliminar refresh token de base de dades
- invalidar token

Flux simple:

1. Eliminar refresh token associat a l'usuari.

4.6 Middleware d'autenticació

Crear middleware:

middleware/authMiddleware.js

4.7 RBAC (Role Based Access Control)

Els rols definits seran:

- client
- admin

Crear middleware:

middleware/roleMiddleware.js

<u>Exemple:</u>

```
module.exports = (...roles) => {

 return (req, res, next) => {

   if (!roles.includes(req.user.role)) {

     return res.status(403).json({ message: "Accés prohibit" });

   }

   next();

 };

};
```

<u>Exemple d’ús:</u>

GET /api/products/admin

```
router.get(

 '/admin',

 authMiddleware,

 roleMiddleware('admin'),

 controller

);
```

Només els **admins** poden accedir.

Referències

- JWT Introduction
  https://jwt.io/introduction
- jsonwebtoken npm
  https://www.npmjs.com/package/jsonwebtoken
- bcryptjs npm
  https://www.npmjs.com/package/bcryptjs
- OWASP Authentication Cheat Sheet
  https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

Apèndixs

A. Possible estructura recomanada del backend

```
src
 ├── controllers
 │    authController.js
 │
 ├── middleware
 │    authMiddleware.js
 │    roleMiddleware.js
 │
 ├── models
 │    User.js
 │
 ├── routes
 │    authRoutes.js
 │
 └── config
     db.js
```

B. Bones pràctiques amb JWT

Access token **curt (10-15 min)**

Refresh token **més llarg (7-30 dies)**

No guardar contrasenyes en text pla

C. Errors comuns

1. **Guardar contrasenyes sense hash**

Incorrecte:

password: "123456"

Correcte:

password: bcrypt.hash(...)

2. **No verificar tokens**

Sempre validar:

jwt.verify()

Lliurable de la Sessió 9

PDF amb totes les explicacions i link a GitHub.